7.10 프로그래밍 연습 문제 정답 (MASM / Irvine32)

★ 1. WriteScaled

```
INCLUDE Irvine32.inc

DECIMAL_OFFSET = 5
.data
decimal_one   BYTE "100123456789765"
decimal_two   BYTE "12345"
decimal_three BYTE "9876543210"
.code
main PROC
    mov  edx,OFFSET decimal_one
    mov  ecx,LENGTHOF decimal_one
    mov  ebx,DECIMAL_OFFSET
    call WriteScaled
    call Crlf
    mov  edx,OFFSET decimal_two
    mov  ecx,LENGTHOF decimal_two
    mov  ebx,2
    call WriteScaled
    call Crlf
    mov  edx,OFFSET decimal_three
    mov  ecx,LENGTHOF decimal_three
    mov  ebx,3
    call WriteScaled
    call Crlf
    exit
main ENDP

; EDX = 숫자 오프셋, ECX = 숫자 길이, EBX = 소수 자릿수
WriteScaled PROC
    pushad
    sub  ecx,ebx          ; 정수부 길이
    mov  edi,ebx          ; 소수부 길이 저장
    mov  esi,edx
    jecxz Frac
IntLoop:
    mov  al,[esi]
    call WriteChar
    inc  esi
    loop IntLoop
Frac:
    mov  al,'.'
    call WriteChar
    mov  ecx,edi
    jecxz Done
FracLoop:
    mov  al,[esi]
    call WriteChar
    inc  esi
    loop FracLoop
Done:
    popad
    ret
WriteScaled ENDP
END main
```

```
1001234567.89765
123.45
9876543.210
```

★★ 2. Extended_Sub

```
INCLUDE Irvine32.inc

; 12바이트(3 dword) 정수 세 쌍 (리틀 엔디안: 낮은 dword가 먼저)
RunTest MACRO x,y
    mov  esi,OFFSET x
    mov  edi,OFFSET y
    mov  ebx,OFFSET diff
    mov  ecx,LENGTHOF x
    call Extended_Sub
    mov  esi,OFFSET diff
    mov  ecx,LENGTHOF diff
    call ShowBig
ENDM
.data
a1 DWORD 0,0,2
b1 DWORD 1,0,0
a2 DWORD 89ABCDEFh,01234567h,0FEDCBA98h
b2 DWORD 11111111h,22222222h,33333333h
a3 DWORD 5,5,5
b3 DWORD 6,5,5
diff DWORD 3 DUP(?)
.code
main PROC
    RunTest a1,b1
```

```
    RunTest a2,b2
    RunTest a3,b3
    exit
main ENDP

; ESI = 피감수, EDI = 감수, EBX = 결과, ECX = dword 개수
Extended_Sub PROC
    pushad
    clc
L1:
    mov  eax,[esi]
    sbb  eax,[edi]
    mov  [ebx],eax
    pushfd                ; CF 보존
    add  esi,4
    add  edi,4
    add  ebx,4
    popfd
    loop L1
    popad
    ret
Extended_Sub ENDP

; ESI = 배열, ECX = dword 개수 (높은 dword부터 출력)
ShowBig PROC
    pushad
    lea  esi,[esi+ecx*4-4]
L1:
    mov  eax,[esi]
    call WriteHex
    mov  al,' '
    call WriteChar
    sub  esi,4
    loop L1
    call Crlf
    popad
    ret
ShowBig ENDP
END main
```

```
00000001 FFFFFFFF FFFFFFFF
CBA98764 DF012345 789ABCDE
FFFFFFFF FFFFFFFF FFFFFFFF
```

★★ 3. PackedToAsc

```
INCLUDE Irvine32.inc

.data
packedNums DWORD 12345678h, 00000042h, 99999999h, 00001000h, 87654321h
buffer     BYTE 9 DUP(?)
.code
main PROC
    mov  esi,OFFSET packedNums
    mov  ecx,LENGTHOF packedNums
L1:
    mov  eax,[esi]
    mov  edx,OFFSET buffer
    call PackedToAsc
    call WriteString
    call Crlf
    add  esi,4
    loop L1
    exit
main ENDP

; EAX = 4바이트 패킹 10진수, EDX = 버퍼 오프셋 (9바이트 이상)
PackedToAsc PROC
    pushad
    mov  ecx,8
L1:
    rol  eax,4            ; 최상위 니블부터 꺼냄
    mov  bl,al
    and  bl,0Fh
    add  bl,'0'
    mov  [edx],bl
    inc  edx
    loop L1
    mov  BYTE PTR [edx],0
    popad
    ret
PackedToAsc ENDP
END main
```

```
12345678
00000042
99999999
00001000
87654321
```

★★ 4. 회전 연산을 이용한 암호화

```
INCLUDE Irvine32.inc

KEYSIZE = 10
.data
key1 SBYTE -2,4,1,0,-3,5,2,-4,-4,6
key2 SBYTE 3,-1,7,-7,2,0,-5,1,8,-2
msg1 BYTE "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
msg2 BYTE "Rotate every byte now"
.code
main PROC
    mov  esi,OFFSET msg1
    mov  ecx,LENGTHOF msg1
    mov  edi,OFFSET key1
    call Encrypt
    mov  ebx,TYPE msg1
    call DumpMem          ; 암호화된 데이터 출력 (ESI, ECX 그대로)

    mov  esi,OFFSET msg2
    mov  ecx,LENGTHOF msg2
    mov  edi,OFFSET key2
    call Encrypt
    mov  ebx,TYPE msg2
    call DumpMem
    exit
main ENDP

; ESI = 메시지, ECX = 길이, EDI = 키 배열(10바이트)
; 음수 = 왼쪽 회전, 양수 = 오른쪽 회전
Encrypt PROC
    pushad
    mov  ebx,0            ; 키 인덱스
L1:
    mov  dl,[edi+ebx]
    push ecx
    mov  cl,dl
    test dl,dl
    js   RotLeft
    ror  BYTE PTR [esi],cl
    jmp  Next
RotLeft:
    neg  cl
    rol  BYTE PTR [esi],cl
Next:
    pop  ecx
    inc  esi
    inc  ebx
    cmp  ebx,KEYSIZE
    jb   KeyOK
    mov  ebx,0            ; 다음 10바이트: 키를 처음부터 다시 정렬
KeyOK:
    loop L1
    popad
    ret
Encrypt ENDP
END main
```

★★★ 5. 소수 (에라토스테네스의 체)

```
INCLUDE Irvine32.inc

LIMIT = 1000
.data
sieve BYTE LIMIT+1 DUP(1)
.code
main PROC
    mov  sieve[0],0
    mov  sieve[1],0
    mov  esi,2
Outer:
    mov  eax,esi
    mul  esi              ; EAX = i*i
    cmp  eax,LIMIT
    ja   Report
    cmp  sieve[esi],0
    je   NextI
    mov  edi,eax          ; j = i*i
Inner:
    mov  sieve[edi],0
    add  edi,esi
    cmp  edi,LIMIT
    jbe  Inner
NextI:
    inc  esi
    jmp  Outer
Report:
    mov  esi,2
    mov  ebx,0            ; 한 줄에 출력한 개수
Show:
    cmp  esi,LIMIT
    ja   Done
    cmp  sieve[esi],0
    je   Skip
```

```
    mov  eax,esi
    call WriteDec
    mov  al,' '
    call WriteChar
    inc  ebx
    cmp  ebx,15
    jb   Skip
    call Crlf
    mov  ebx,0
Skip:
    inc  esi
    jmp  Show
Done:
    call Crlf
    exit
main ENDP
END main
```

```
2 3 5 7 11 13 ... 983 991 997   (총 168개)
```

★★★ 6. 최대공약수 (GCD)

```
INCLUDE Irvine32.inc

.data
pairs   SDWORD 48,18, -35,14, 17,5, 0,9, 270,192, -84,-36, 100,0
strGcd  BYTE "GCD(",0
strComma BYTE ", ",0
strEq   BYTE ") = ",0
.code
main PROC
    mov  esi,OFFSET pairs
    mov  ecx,(LENGTHOF pairs) / 2
L1:
    mov  edx,OFFSET strGcd
    call WriteString
    mov  eax,[esi]
    call WriteInt
    mov  edx,OFFSET strComma
    call WriteString
    mov  eax,[esi+4]
    call WriteInt
    mov  edx,OFFSET strEq
    call WriteString
    mov  eax,[esi]
    mov  ebx,[esi+4]
    call GCD
    call WriteDec
    call Crlf
    add  esi,8
    loop L1
    exit
main ENDP

; EAX = x, EBX = y  →  EAX = GCD(x, y)
GCD PROC
    push ebx
    push edx
    cmp  eax,0
    jge  L1
    neg  eax              ; x = abs(x)
L1:
    cmp  ebx,0
    jge  L2
    neg  ebx              ; y = abs(y)
L2:
    cmp  ebx,0
    je   Done             ; y = 0 이면 결과는 x
Top:
    xor  edx,edx
    div  ebx              ; EDX = x % y
    mov  eax,ebx          ; x = y
    mov  ebx,edx          ; y = n
    cmp  ebx,0
    jg   Top
Done:
    pop  edx
    pop  ebx
    ret
GCD ENDP
END main
```

```
GCD(+48, +18) = 6
GCD(-35, +14) = 7
GCD(+17, +5) = 1
GCD(+0, +9) = 9
GCD(+270, +192) = 6
GCD(-84, -36) = 12
GCD(+100, +0) = 100
```

★★★ 7. BitwiseMultiply

```
INCLUDE Irvine32.inc

.data
nums DWORD 5,7, 123,456, 1000,1000, 65535,65537
.code
main PROC
    mov  esi,OFFSET nums
    mov  ecx,(LENGTHOF nums) / 2
L1:
    mov  eax,[esi]
    mov  ebx,[esi+4]
    call BitwiseMultiply
    call WriteDec
    call Crlf
    add  esi,8
    loop L1
    exit
main ENDP

; EAX = 피승수, EBX = 승수  →  EAX = 곱 (시프트와 덧셈만 사용)
BitwiseMultiply PROC
    push ebx
    push edx
    mov  edx,eax          ; 피승수(왼쪽으로 계속 시프트)
    xor  eax,eax          ; 곱 = 0
L1:
    cmp  ebx,0
    je   Done
    shr  ebx,1            ; 승수의 최하위 비트 → CF
    jnc  NoAdd
    add  eax,edx          ; 비트가 1이면 시프트된 피승수를 더함
NoAdd:
    shl  edx,1
    jmp  L1
Done:
    pop  edx
    pop  ebx
    ret
BitwiseMultiply ENDP
END main
```

```
35
56088
1000000
4294967295
```

★★★ 8. AddPacked (임의 크기)

```
INCLUDE Irvine32.inc

RunTest MACRO x,y
    mov  esi,OFFSET x
    mov  edi,OFFSET y
    mov  edx,OFFSET sum
    mov  ecx,LENGTHOF x
    call AddPacked
    mov  esi,OFFSET sum
    mov  ecx,LENGTHOF x
    call ShowPacked
ENDM
.data
a4  BYTE 78h,56h,34h,12h
b4  BYTE 22h,43h,65h,87h
a8  BYTE 56h,34h,12h,90h,78h,56h,34h,12h
b8  BYTE 8 DUP(99h)
a16 BYTE 12h,90h,78h,56h,34h,12h,90h,78h,56h,34h,12h,90h,78h,56h,34h,12h
b16 BYTE 98h,10h,32h,54h,76h,98h,10h,32h,54h,76h,98h,10h,32h,54h,76h,98h
sum BYTE 17 DUP(?)        ; 크기 + 1 (마지막 올림 바이트)
.code
main PROC
    RunTest a4,b4
    RunTest a8,b8
    RunTest a16,b16
    exit
main ENDP

; ESI = 첫 번째 수, EDI = 두 번째 수, EDX = 합(크기+1 바이트), ECX = 바이트 수
; 수는 리틀 엔디안(낮은 자리 바이트가 먼저)
AddPacked PROC
    pushad
    clc
L1:
    mov  al,[esi]
    adc  al,[edi]
    daa
    mov  [edx],al
    inc  esi              ; INC는 CF를 바꾸지 않음
    inc  edi
    inc  edx
    loop L1
    mov  BYTE PTR [edx],0
    adc  BYTE PTR [edx],0 ; 마지막 올림
```

```
    popad
    ret
AddPacked ENDP

; ESI = 합 버퍼, ECX = 바이트 수 (올림 바이트 → 높은 바이트부터 출력)
ShowPacked PROC
    pushad
    mov  ebx,TYPE BYTE
    movzx eax,BYTE PTR [esi+ecx]
    cmp  eax,0
    je   L1
    call WriteHexB
L1:
    movzx eax,BYTE PTR [esi+ecx-1]
    call WriteHexB
    loop L1
    call Crlf
    popad
    ret
ShowPacked ENDP
END main
```

```
0100000000
011234567890123455
01111111110111111111101111111110110
```